

UNIVERSIDADE
DOS AÇORES

LICENCIATURA EM INFORMÁTICA

EDIÇÃO 2020/2021

SISTEMAS DISTRIBUÍDOS

3º ANO / 2º SEMESTRE

Elaborado por:

Docentes:

Pedro Melo

Manuel Cabral

Salif Faustino

Francisco Martins

José Cascalho

18/06/2021

Resumo

A indústria dos jogos é hoje em dia um ramo importante da engenharia de software (e da economia em geral). Com orçamentos ao nível dos filmes mais caros de *Hollywood*, os jogos apresentam requisitos extremos que introduzem desafios sem precedentes à construção de *software*, em particular, de sistemas distribuídos. Além do grafismo, que está ao nível da produção cinematográfica, e do desempenho, que pressiona a indústria de software e de *hardware* aos limites, nos tempos recentes tem-se assistido ao domínio dos jogos *online* multi-jogador, em que pessoas de todo o mundo formam equipas para se defrontarem. São exemplos desta tendência jogos como o *League of Legends*, o *Counter Strike* ou até alguns mais clássicos que não tinha enveredado inicialmente por este caminho, como o caso do *FIFA*. Estes, não só alcançam gráficos de extrema qualidade, como também utilizam muito a inteligência artificial, que faz com que partes do jogo ganhem “vida própria”, como também nos permitem formar equipas com jogadores dos quatro cantos do globo que cooperam em tempo real para levar a sua equipa à vitória.

O que é proposto neste projeto é a construção de um jogo, multi-jogador, distribuído, mas em termos muito mais modestos. O objetivo irá centrar-se em torno dos tópicos da disciplina de Sistemas Distribuídos, em particular, na comunicação dos jogadores com o servidor que aloja o jogo, da partilha de parte do estado global do jogo entre os seus jogadores e, por último, na atualização, em tempo real, da interface de utilizador dos diversos jogadores.

Glossário

Python - é uma linguagem de programação de alto nível, interpretada de script, imperativa, orientada a objetos, funcional e tipifica.

JSON - é um formato compacto, de padrão aberto independente, de troca de dados simples e rápida (parsing) entre sistemas, especificado por Douglas Crockford em 2000, que utiliza texto legíveis a humanos, no formato chave-valor (natureza auto-descritiva).

Pygame – é um *package* da linguagem *Python* que permite programar vídeo jogos e contém módulos gráficos de computador e de som.

Tkinter – é um *package* da linguagem *Python* que permite criar janelas, botões e outros elementos interativos que podem ser programados para executar funções

Exclusão mútua - é uma propriedade utilizada na programação concorrente que serve para garantir que dois **processos ou threads** não tenham acesso simultâneo a um recurso partilhado (**secção crítica**), ou seja, garante que se uma secção crítica é executada por uma *thread* ou processo, outras tarefas serão impedidas de fazer o mesmo.

Fila (Queue) – é uma estrutura de dados que adota a política **FIFO** (*First in First out* - o primeiro a entrar é o primeiro a sair).

ZeroMQ - é uma biblioteca de mensagens assíncronas, destinada ao uso em sistemas distribuídos ou concorrentes.

Índice

Resumo	2
Glossário	1
1. Introdução	3
2. Análise e Planeamento	4
2.1 Descrição do jogo	4
2.2 Pacotes utilizados	5
2.3 Análise dos casos de uso	5
2.4 Diagrama de domínio	7
3. Implementação e organização do jogo	7
3.1 Fila de jogadores	7
3.2 Variável que guarda o estado atual	8
3.3 Tabuleiro	8
4. Como garantir a exclusão mútua	9
5. Futuras Melhorias	10
Conclusão	11

1. Introdução

Este projeto foi realizado no âmbito da cadeira de Sistemas Distribuídos, do terceiro ano do curso de Informática. Tem como objetivo o desenvolvimento do jogo **tetris** multijogador no qual os jogadores comunicam com o jogo através de um servidor. Este trabalho pretende reforçar a aprendizagem da linguagem *Python* e os conhecimentos adquiridos na disciplina relativamente à comunicação num sistema distribuído centralizado.

2. Análise e Planeamento

2.1 Descrição do jogo

O jogo consiste em movimentar peças, chamadas *tetriminos*, num tabuleiro vertical. As peças surgem do topo de forma aleatória e caem até tocarem na base do tabuleiro ou em alguma peça já colocada previamente. À medida que as peças caem, podem ser rodadas para a direita ou para a esquerda. Ao tocarem na base vão preenchendo linhas, que ao estarem totalmente preenchidas são eliminadas.

Quando um jogador pretende iniciar o seu jogo é necessário que o servidor esteja previamente inicializado através do ficheiro `__main__` que se encontra no seguinte diretório `\dt_server\main`.

Após inicializar o servidor(jogo), o jogador indica que pretende jogar através do ficheiro `__main__` que se encontra no seguinte diretório: `\dt_client\main`.

De seguida é apresentada uma janela com um botão *connect*, no qual o cliente pode indicar o seu nome para poder aceder a uma subjanela. Quando o cliente insere o nome e prime o botão *connect*, uma conexão com o servidor é iniciada e é criado um objeto do tipo `Player` com os seguintes atributos: nome, número de linhas e um booleano indicando a sua vez.

Apresentada a subjanela, o jogador poderá então iniciar o seu jogo visualizando as peças a cair e com possibilidade de as mover para a esquerda ou para a direita com o objetivo de completar o máximo de linhas que conseguir. Para além disso também pode apressar a queda da peça.

Sempre que um jogador elimina uma linha, o atributo número de linhas é incrementado em uma unidade. No caso de o jogador não conseguir eliminar as linhas até o *gameboard* estar preenchido até ao topo, o jogo termina e são apresentados detalhes acerca da prestação do jogador, e para além disso informar a possibilidade de recomeçar o jogo. Se estiverem dois jogadores é apresentado também o nome do jogador vencedor.

2.2 Pacotes utilizados

Para o desenvolvimento do projeto foram utilizados os seguintes pacotes:

- **Interface**

A interface do jogo foi feita recorrendo ao *package pygame*, na versão 1.9.6. Este *package* permite programar vídeo jogos e contém módulos gráficos de computador e de som.

As janelas que servem para a interação do jogador com o jogo (botões e campos de texto) foram implementadas com o *package Tkinter* que permite criar janelas, botões e outros elementos interativos que podem ser programados para executar funções.

- **Comunicação**

A comunicação foi feita através de *sockets ZMQ* do tipo *Request-Reply* e foi também utilizado o mecanismo de *locks* para alcançar a exclusão mútua e evitar *deadlocks*, ao que chamamos em português de concorrência ou condições de corrida.

2.3 Análise dos casos de uso

Depois de ler o enunciado, foram elaborados diagramas de sequência que respondem às necessidades dos casos de usos propostos:

- caso de uso **entrada de novo jogador** - Um jogador indica ao jogo que pretende entrar no jogo. O sistema pede o seu nome (uma string). O sistema verifica que não existe outro jogador com o mesmo nome e adiciona-o (e mantém-se ligado)

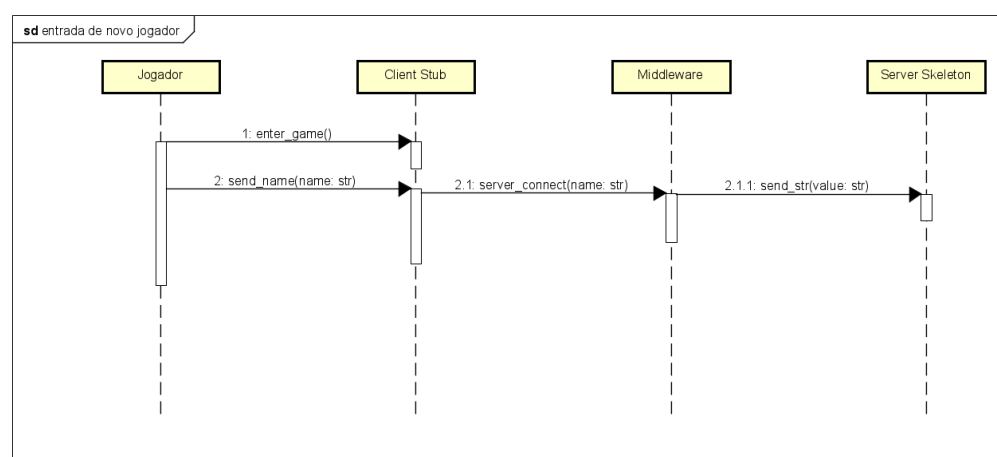


Figura 1- Caso de uso entrada de novo jogador

- caso de uso **jogar** - Este é o caso de uso central do jogo. Quando o jogador indica que pretende jogar o jogo é lhe apresentada o tabuleiro vazio e uma peça no topo que foi escolhida pelo jogo de forma aleatória. O jogador pode indicar ao jogo que pretende rodar a peça para a esquerda, para a direita ou mover para baixo.

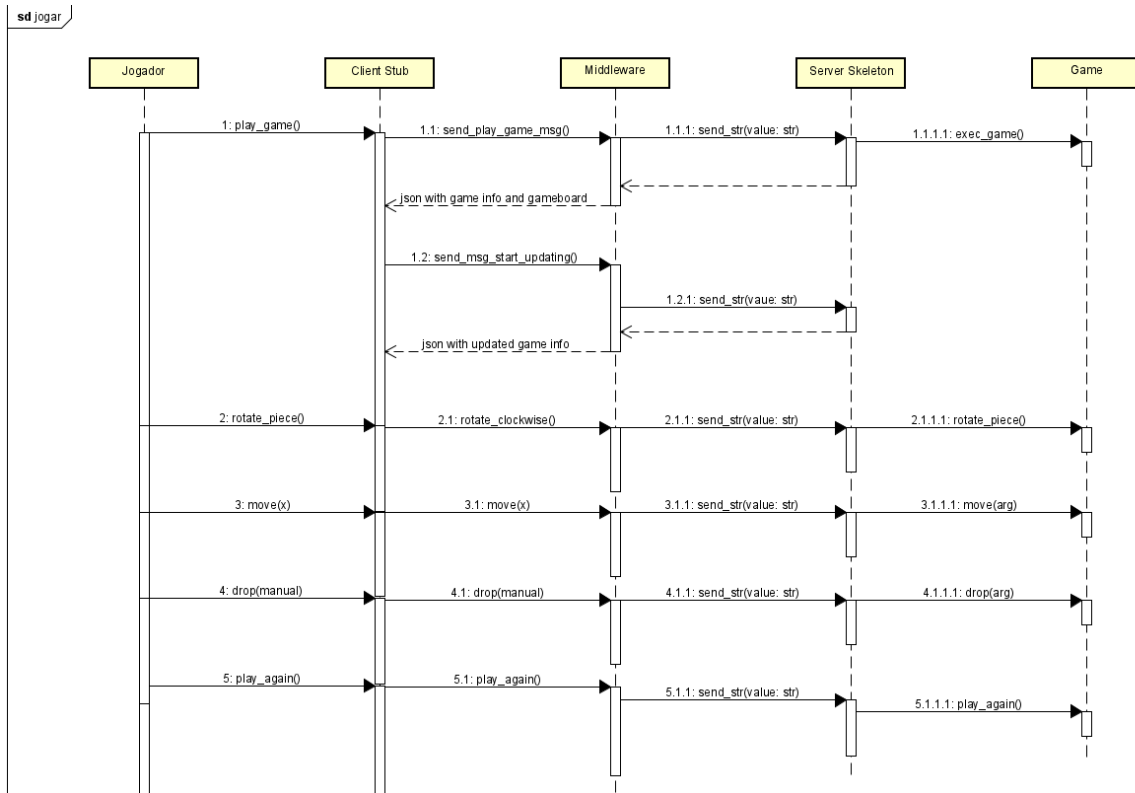
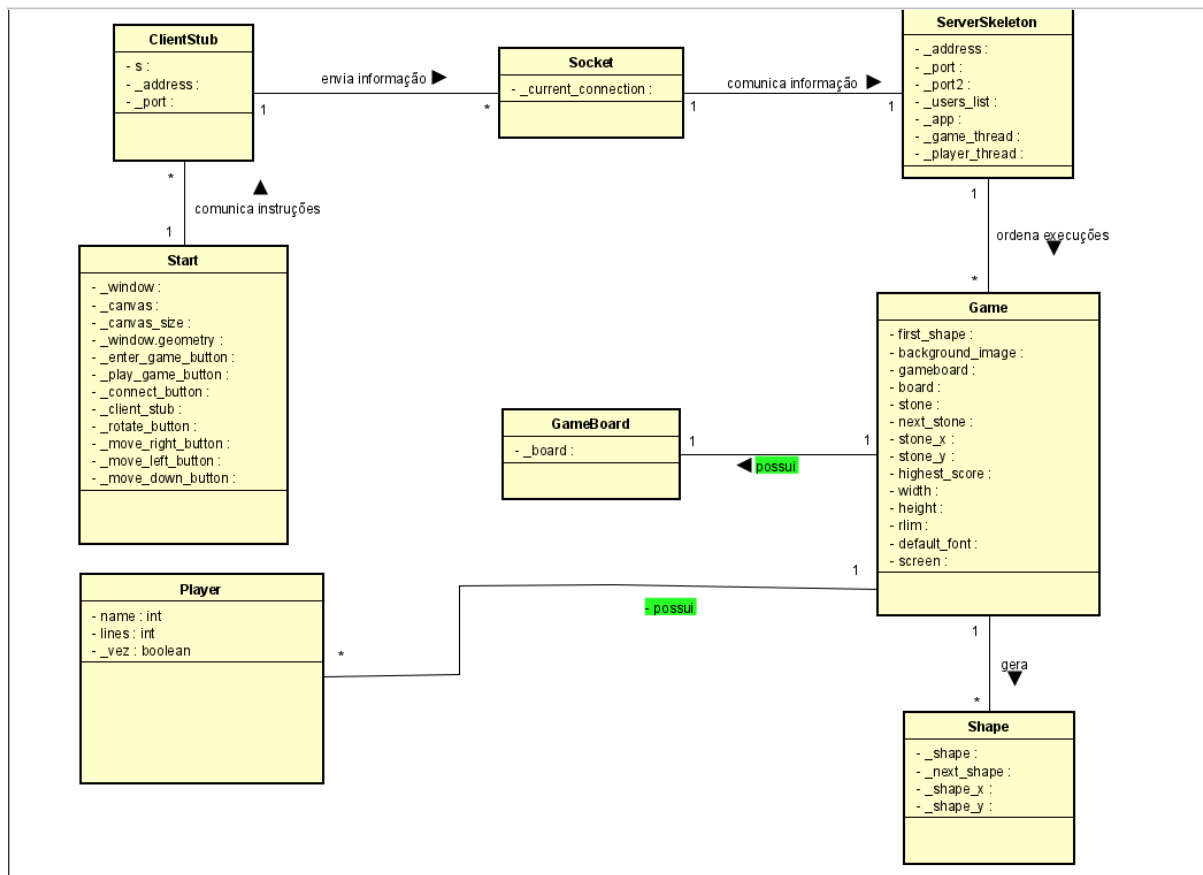


Figura 2- caso de uso jogar

Deste modo, é importante também salientar que os casos de uso foram também feitos na versão simplificada em que apenas temos o cliente(jogador) e o servidor(jogo). Acharmos ideal apresentar esta versão porque permite perceber o mecanismo por trás do envio de um pedido do jogador(cliente) e a resposta por parte do servidor(jogo).

2.4 Diagrama de domínio



3. Implementação e organização do jogo

A implementação do jogo foi feita recorrendo ao paradigma de programação centrada em objetos, obedece todas as regras já aprendidas.

No que diz respeito à estrutura, o projeto organizou-se da seguinte forma:

- A parte lógica do jogo foi construída no ficheiro `game.py`, dentro do servidor, e os seus aspetos mais importantes são:
 - A fila de jogadores
 - A variável que guarda o jogador atual (aquele que tem a vez de jogar)
 - O tabuleiro

3.1 Fila de jogadores

Este atributo da classe *Game* é uma *Queue* que foi implementada pelos elementos do grupo e suporta operações como *add*, *peek*, *pop*, *clear*, *is_empty*, etc.

É constituída por objetos do tipo *Player*, classe esta que tem apenas 3 atributos: nome do jogador, nº de linhas completadas e um atributo booleano que indica se é a sua vez de jogar.

Sempre que um jogador coloca uma peça, o jogo passa a vez ao próximo jogador na fila, portanto faz o *pop* ao jogador que acabou de jogar, e de seguida faz o *add* para adicioná-lo de volta à fila no último lugar. Caso o jogador tenha completado uma linha a sua pontuação é incrementada em 1.

3.2 Variável que guarda o estado atual

Esta variável guarda um objeto do tipo *Player*. É sincronizada com os clientes para que possam atualizar o seu estado no que toca à sua vez de jogar e assim perceberem se é a sua vez ou não.

Este estado é importante para auxiliar na exclusão mútua, uma vez que existe uma condição que só deixa um jogador enviar um pedido caso seja a sua vez de jogar.

3.3 Tabuleiro

Para o tabuleiro foi criada uma classe *Gameboard* que contém métodos para rodar peças, verificar colisões, remover filas e juntar matrizes. É sincronizado com os clientes para que possam todos observar o mesmo jogo.

Em relação à comunicação, temos dois componentes importantes sendo eles o *ClientStub.py* e o *ServerSkeleton.py*. Estes dois componentes estão em constante comunicação a partir do momento que um jogador clica em “*Connect*” depois de introduzir o seu nome de utilizador.

Logo após ter clicado em “*Connect*”, o Cliente manda um pedido para estabelecer conexão que passa pelo seu *stub*. De seguida esse *stub* cria o *socket* do cliente e estabelece um canal de comunicação com o servidor e logo a seguir envia-lhe o pedido para adicionar o jogador.

Portanto o *Server Skeleton* recebe o pedido e processa-o ao chamar o método do jogo que verifica se o nome é válido (não pode ser igual ao de outro jogador). Caso o nome seja válido o método retorna o jogador (objeto do tipo *Player*), caso contrário retorna *None*. Neste último caso, o que acontece depois é que o *ClientStub* envia novamente um pedido para adicionar o jogador, mas desta vez com o nome “*Nome_do_Jogador2*”, ou seja, acrescenta um 2 no final do nome. Esta abordagem foi inspirada no contexto de criação de ficheiros com o mesmo nome no *Windows* (ficheiros com o mesmo nome ficam nomeados “*Nome_do_Ficheiro-Cópia*”).

Agora, a parte mais importante do projeto acontece: a **sincronização** do servidor com os seus clientes. Após clicar em “*Play Game*”, o cliente começa a receber informação do servidor. A abordagem implementada tem como objetivo enviar JSONs do servidor para o cliente no método *send_data* do *ServerSkeleton*. Dentro do JSON temos *strings*, inteiros, *floats* e booleanos. As *strings* na sua maioria são instruções para mais tarde serem executadas funções do lado do cliente no seu *stub* recorrendo à função *eval*.

Apenas a parte gráfica, bem como o estado do jogo, são enviados aos clientes de modo a reduzir o consumo de recursos. Assim é como se o cliente estivesse a jogar um jogo no browser como os jogos de *Flash Player*. Não é necessário download, apenas é preciso receber a parte gráfica, enviar comandos e ter um canal de comunicação.

Para que a comunicação seja constante, no *ServerSkeleton* existe um ciclo *while* que contém um *lock* (ferrolho) para tornar exclusiva a secção de recebimento de pedidos. No lado do *ClientStub*, temos um *Timer* do pacote *Threading* do Python. Escolhemos isto para evitar termos outro ciclo *while* e assim tornar o programa mais eficiente.

4. Como garantir a exclusão mútua

Antes demais é preciso perceber o que é exclusão mútua. Exclusão mútua (*mutual exclusion*) é uma propriedade utilizada na programação concorrente que serve para garantir que dois processos ou *threads* não tenham acesso simultâneo a um recurso partilhado (secção crítica), ou seja, garante que se uma secção crítica é executada por uma *thread* ou processo, outras tarefas serão impedidas de fazer o mesmo (atomicidade).

Considerando que o nosso projeto tem como base um sistema centralizado, que funciona com um processo eleito como líder ou coordenador (servidor) e sempre que um processo deseja aceder a um recurso compartilhado envia um pedido ao líder informando qual é o recurso que pretende aceder. Se nenhum outro processo estiver a aceder a esse recurso, o líder envia de volta concebendo a permissão. Se o recurso estiver a ser utilizado por outro processo, o líder bloqueia e guarda os pedidos numa fila com a ordem de chegada, quando o processo termina são permitidos os pedidos que estiverem na fila, um de cada vez.

Neste projeto a forma como garantimos a exclusão mútua é a seguinte:

1. Quando um jogador se conecta ao servidor é adicionado numa fila.
2. O jogador que estiver à frente da fila tem prioridade para jogar.
3. Quando é a vez do jogador com prioridade em jogar, o jogador é removido da fila utilizando a função **pop()** que remove o primeiro elemento da fila.
4. Enquanto um jogador estiver a jogar, é utilizado o mecanismo dos **locks()** das *threads* para evitar que outros jogadores intervenham em simultâneo.
5. Assim que o jogador terminar, no caso de preencher uma linha ou a peça estiver na posição limite do tabuleiro é adicionado no final da fila utilizando a função **add()**(que adiciona no final da fila) e consequente deixa de ser a sua vez de jogar ficando assim impedido de manipular a peça seguinte.
6. O jogador seguinte repete os passos 3,4 e 5.

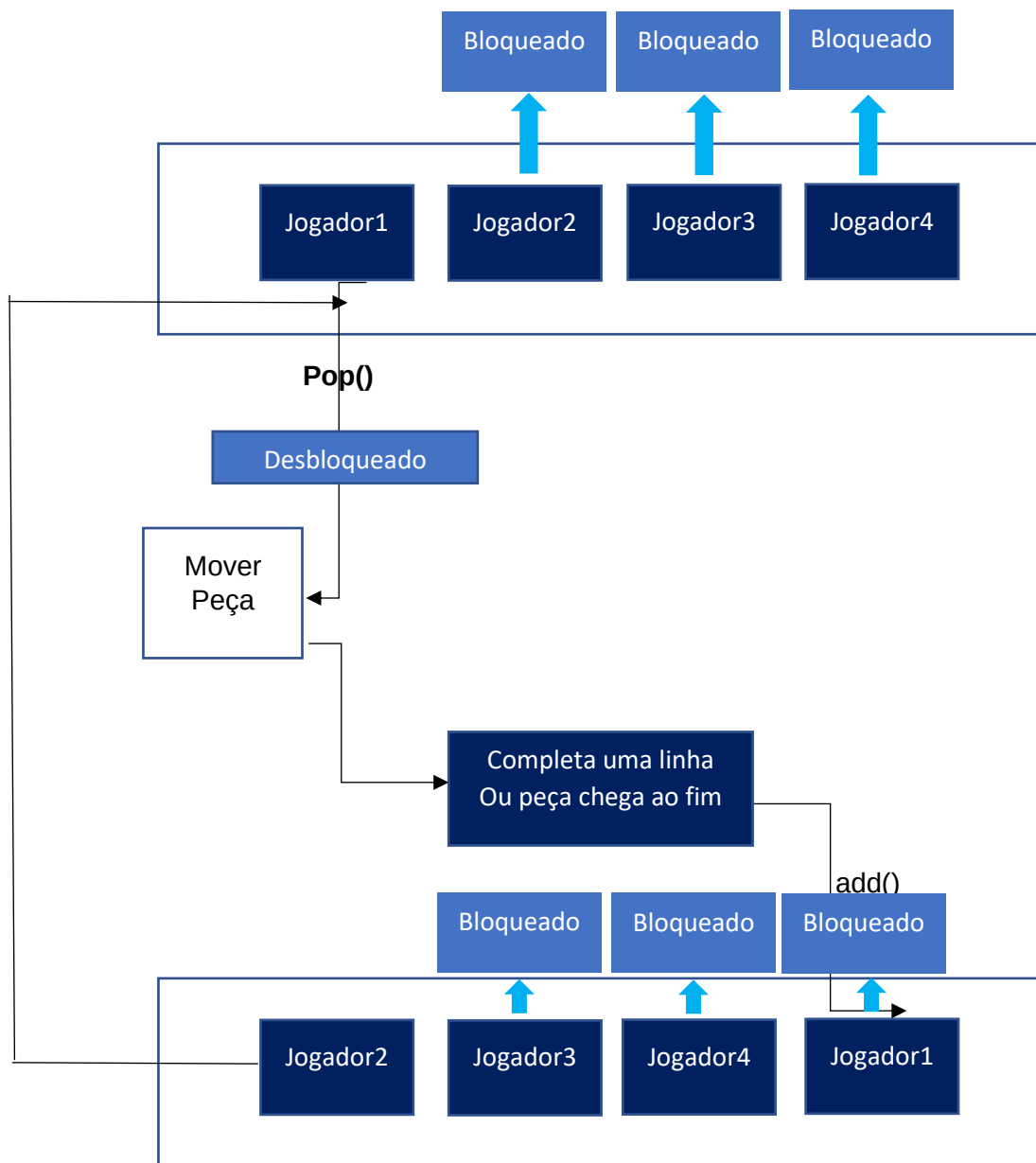


Figura 3 - Como garantir a exclusão mútua

5. Futuras Melhorias

Tendo em conta que a solução passa por implementar um sistema centralizado, este apresenta desvantagens. Por exemplo o facto de o servidor ser o único ponto de falha quando para de funcionar os jogadores não terão como continuar o jogo.

Uma futura melhoria passaria por implementar um sistema descentralizando, onde todo processamento é executado de forma distribuída, não é definido um servidor como na arquitetura centralizada. Os recursos partilhados são replicados nos sistemas de cada cliente (jogador). Cada jogador teria uma instância completa do jogo executando na sua máquina. A arquitetura descentralizada é mais tolerante a falhas, pois as informações são replicadas e não há um servidor central. A ausência do servidor também contribui para a escalabilidade do

sistema já que não há um ponto central de processamento. Por outro lado, sincronizar e manter a consistência do estado distribuído são tarefas complexas.

Conclusão

De um modo geral, conseguimos alcançar os objetivos pretendidos embora tenha havido uma abordagem diferente relativamente ao controlo das peças, visto que o professor pensou em ter várias peças a caírem em simultâneo onde cada jogador controlaria e via a sua peça. A nossa solução passa por definir a vez de cada jogador como é feito nos jogos tradicionais de tetris multijogador.

O projeto proporcionou uma experiência na utilização do protocolo *zmq*, visto que fizemos várias alterações na maneira como estávamos a estabelecer a comunicação entre cliente e servidor.

Inicialmente utilizamos uma abordagem de *push/pull* visto que a janela estava do lado do servidor(jogo). O jogador(cliente) fazia *push* dos eventos e o servidor(jogo) o *pull* para executar as funções que o jogador pretendia. Desta forma, o servidor não enviava nenhuma resposta.

A abordagem acima mostrou-se ser um problema quando foi necessário implementar a segunda parte do projeto, que consistia em múltiplos jogador em que cada jogador tinha que controlar sua peça na sua janela. Nesta abordagem o cliente faz um pedido ao servidor e o servidor aplica as regras necessárias e devolve a resposta ao jogador e este por sua vez faz o render. Para ultrapassar o problema tivemos que refazer a maneira como a comunicação era feita aplicando apenas o *zmq* do tipo *Request-Reply*.